MINI-PROJET

Création et mise en ligne d'une application web en architecture MVC

Application de gestion d'une bibliothèque — « BiblioNet »

Établissement	.....
Filière / Niveau	.....
Module	Développement web — PHP / MySQL (architecture MVC)
Enseignant	Charly OBAME
Travail	En groupe de 3 étudiants
Durée	4 jours (J1 à J4) — soutenance l'après-midi du J4
Date de démarrage	..... / ..... / 20.....

Ce document décrit le contexte, le cahier des charges, l'architecture attendue, la répartition des tâches, le planning jour par jour, la procédure de mise en ligne, les livrables et la grille d'évaluation.

1. Contexte

La bibliothèque d'un établissement scolaire gère aujourd'hui ses ouvrages et ses prêts sur des cahiers. Il est difficile de savoir si un livre est disponible et qui l'a emprunté. La direction souhaite une **application web simple**, accessible en ligne, pour gérer les livres, les adhérents et les emprunts.

Votre équipe de **3 développeurs** dispose de **4 jours** pour concevoir cette application en respectant l'**architecture MVC (Modèle – Vue – Contrôleur)**, puis la **mettre en ligne** à une adresse accessible depuis Internet.

2. Objectifs pédagogiques

À l'issue du mini-projet, l'étudiant doit être capable de :

- comprendre et appliquer le patron d'architecture MVC sans framework ;
- mettre en place un point d'entrée unique (front controller) et un routeur simple ;
- accéder à une base MySQL de façon sécurisée avec PDO et des requêtes préparées ;
- gérer une authentification par session ;
- travailler à plusieurs sur un même code avec Git et GitHub ;
- déployer une application PHP/MySQL sur un hébergement et la rendre accessible par une URL publique.

3. Cahier des charges (périmètre réduit à 4 jours)

3.1 Acteurs

Acteur	Rôle
Bibliothécaire (connecté)	Gère les livres, les adhérents, enregistre les emprunts et les retours.
Visiteur (non connecté)	Consulte le catalogue et recherche un livre.

3.2 Fonctionnalités

Module	Fonctionnalités	Priorité
Authentification	Connexion / déconnexion d'un compte bibliothécaire. Mot de passe haché (password_hash / password_verify). Pages de gestion protégées par la session.	Obligatoire
Livres	Lister, ajouter, modifier, supprimer (CRUD). Champs : titre, auteur, catégorie, année, nombre d'exemplaires.	Obligatoire
Adhérents	CRUD des adhérents (nom, prénom, classe, téléphone).	Obligatoire
Emprunts	Enregistrer un emprunt (livre + adhérent), enregistrer le retour, refuser l'emprunt si aucun exemplaire n'est disponible, lister les emprunts en cours.	Obligatoire
Catalogue public	Liste des livres avec recherche par titre ou auteur et mention « Disponible / Indisponible ».	Obligatoire
Bonus	Tableau de bord chiffré, liste des retards (au-delà de 14 jours), pagination, jeton CSRF, export CSV des emprunts, HTTPS activé.	Facultatif

3.3 Contraintes techniques

- Langage : **PHP 8.x** orienté objet, **sans framework** (pas de Laravel ni Symfony) ;
- Base de données : **MySQL / MariaDB**, accès via **PDO** et requêtes préparées uniquement ;
- Interface : HTML5 et CSS3, **Bootstrap fortement conseillé** pour gagner du temps ;
- Aucune requête SQL dans les vues ; aucun HTML dans les modèles ;
- Toutes les données affichées sont échappées avec htmlspecialchars (anti-XSS) ;
- Code versionné sur un dépôt **GitHub** commun ; chaque membre commite son propre travail.

4. Architecture MVC attendue

4.1 Principe

Couche	Responsabilité	Exemple
Modèle (Model)	Accès aux données et règles métier. Communique avec la base via PDO.	Livre::all(), Emprunt::create()
Vue (View)	Affichage HTML uniquement. Reçoit des données déjà préparées.	livres/index.php, livres/form.php
Contrôleur (Controller)	Reçoit la requête, appelle le modèle, choisit la vue.	LivreController::store()
Routeur / Front controller	Point d'entrée unique (public/index.php) qui dirige chaque URL vers la bonne méthode de contrôleur.	/livres/edit/5 → LivreController::edit(5)

4.2 Arborescence imposée

```
biblionet/
├── app/
│   ├── controllers/      AuthController.php, LivreController.php,
│   │                     AdherentController.php, EmpruntController.php,
│   │                     CatalogueController.php
│   ├── models/           User.php, Livre.php, Adherent.php, Emprunt.php
│   ├── views/
│   │   ├── layouts/      main.php (en-tête, menu, pied de page)
│   │   ├── livres/       index.php, form.php
│   │   ├── adherents/    index.php, form.php
│   │   ├── emprunts/     index.php, form.php
│   │   └── catalogue/    index.php
│   └── core/             Router.php, Controller.php, Database.php
├── config/
│   └── config.php        paramètres de connexion (lus depuis .env)
├── public/               ← SEUL dossier exposé au web
│   ├── index.php         front controller
│   ├── .htaccess         réécriture d'URL
│   └── assets/           css/, js/
├── database/
│   └── biblionet.sql     script de création + données de test
├── .env.example
├── .gitignore            (exclut .env)
└── README.md
```

4.3 Exemple de flux d'une requête

L'utilisateur ouvre **/livres** → le fichier **.htaccess** redirige vers **public/index.php** → le **Router** appelle **LivreController::index()** → le contrôleur demande les données à **Livre::all()** → il transmet le résultat à la vue **views/livres/index.php** → la page HTML est renvoyée au navigateur.

```
# public/.htaccess
RewriteEngine On
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule ^(.*)$ index.php?url=$1 [QSA,L]
```

5. Modèle de données (4 tables)

Table	Champs
users	id, nom, email (unique), password
livres	id, titre, auteur, categorie, annee, nb_exemplaires
adherents	id, nom, prenom, classe, telephone
emprunts	id, livre_id (FK), adherent_id (FK), date_emprunt, date_retour (NULL tant que le livre n'est pas rendu)

Règle métier : *exemplaires disponibles = nb_exemplaires − nombre d'emprunts du livre dont date_retour est NULL*. Si le résultat vaut 0, le livre est indisponible et l'emprunt est refusé.

6. Répartition des rôles dans le groupe

Chaque membre est responsable d'un domaine, mais tous doivent comprendre l'ensemble du code.

Membre	Responsabilités principales
Membre 1 — Socle & déploiement	Arborescence, Database (PDO), Router, Controller de base, layout Bootstrap, authentification, dépôt GitHub, mise en ligne.
Membre 2 — Livres & catalogue	Modèle, contrôleur et vues des livres (CRUD), catalogue public avec recherche et disponibilité.
Membre 3 — Adhérents & emprunts	CRUD des adhérents, enregistrement des emprunts et retours, règle de disponibilité, script SQL et données de test.

7. Planning sur 4 jours

Jour	Matin	Après-midi	À présenter en fin de journée
J1 Analyse & socle	Lecture du sujet, répartition, MCD/MLD, maquettes rapides des écrans, création du dépôt GitHub.	Script SQL + données de test ; socle MVC (Router, Database, layout) ; page d'accueil fonctionnelle.	Socle MVC qui tourne en local, base créée
J2 Fonctionnalités 1	M1 : authentification. M2 : CRUD livres. M3 : CRUD adhérents.	Suite des CRUD, intégration sur la branche principale, premiers tests croisés.	Connexion + CRUD livres et adhérents opérationnels
J3 Fonctionnalités 2 & mise en ligne	M2 : catalogue public + recherche. M3 : emprunts et retours. M1 : préparation de l'hébergement.	Mise en ligne d'une première version, correction des bugs liés au serveur.	Application accessible en ligne
J4 Finalisation & soutenance	Tests complets en ligne, corrections, bonus éventuels, README et rapport. **Remise des livrables à 12h00.**	Soutenance de chaque groupe (15 min).	Version finale + soutenance

Conseil : mettez l'application en ligne **dès le J3**. Les problèmes de déploiement (chemins, .htaccess, identifiants de base) prennent toujours plus de temps que prévu.

8. Mise en ligne de l'application

L'application doit être accessible depuis Internet à une URL publique. Pour tenir les délais, l'**hébergement mutualisé** est recommandé (cPanel ou équivalent : hébergeur local ou offre gratuite comme InfinityFree ou AlwaysData). Un déploiement sur VPS est accepté pour les groupes avancés.

8.1 Procédure type

1. Créer la base de données et l'utilisateur MySQL depuis le panneau d'hébergement.
2. Importer le fichier **database/biblionet.sql** via phpMyAdmin.
3. Transférer le projet (FTP avec FileZilla, ou **git clone** si SSH disponible).
4. Faire pointer la racine du site vers le dossier **public/**. Si ce n'est pas possible, placer un .htaccess à la racine redirigeant vers public/.
5. Créer le fichier **.env** sur le serveur avec les identifiants de production (jamais envoyé sur GitHub).
6. Désactiver l'affichage des erreurs en production (display_errors = Off).
7. Tester toutes les fonctionnalités en ligne et créer un compte de démonstration pour le jury.

8.2 Exemple de fichier .env

```
APP_ENV=production
APP_URL=https://biblionet.mondomaine.com
DB_HOST=localhost
DB_NAME=biblionet
DB_USER=biblio_user
DB_PASS=********
```

9. Livrables (J4 à 12h00)

- **Lien du dépôt GitHub** partagé avec l'enseignant, avec un README expliquant l'installation locale ;
- **URL de l'application en ligne** + identifiants du compte de démonstration ;
- **Script SQL** avec un jeu de données de test (au moins 15 livres et 10 adhérents) ;
- **Rapport court** de 4 à 6 pages (PDF) : présentation, MCD, architecture MVC, répartition des tâches, procédure de mise en ligne, difficultés rencontrées, captures d'écran ;
- **Soutenance** de 15 minutes : démonstration en ligne (8 min) + questions à chaque membre (7 min).

10. Grille d'évaluation (sur 20)

Critère	Détail	Points
Respect du MVC	Séparation des couches, routeur, front controller, arborescence.	4
Fonctionnalités	Modules obligatoires opérationnels et règle de disponibilité respectée.	6
Sécurité et qualité du code	PDO préparé, mots de passe hachés, échappement, code lisible.	2
Mise en ligne	Application accessible en ligne et fonctionnelle, .env, erreurs masquées.	3
Git / travail d'équipe	Commits des 3 membres, répartition visible, README.	1,5
Rapport	Clarté et complétude.	1,5
Soutenance	Démonstration fluide, chaque membre explique sa partie et celle des autres.	2
Bonus	Fonctionnalités facultatives réalisées (jusqu'à +2, note plafonnée à 20).	+2
Total		**20**

11. Consignes et règles

- Chaque membre doit pouvoir expliquer n'importe quelle partie du code lors de la soutenance ; la note peut être individualisée.
- Le plagiat ou la copie d'un projet existant entraîne la note de 0 pour tout le groupe.
- L'usage d'outils d'IA est autorisé pour s'aider, mais le code doit être compris et justifié.
- Livrables remis après J4 12h00 : pénalité de 1 point par heure de retard.
- Une application non accessible en ligne le jour de la soutenance perd la totalité des points « Mise en ligne ».

Bon courage à toutes et à tous !